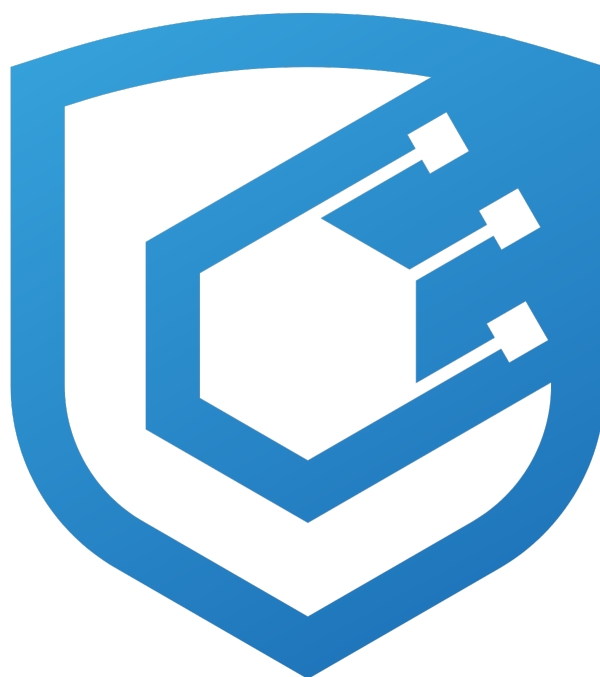




Securitize Stellar DSToken Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[0xFlint](#)

[Carrotsmugger](#)

September 23, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Access Control	3
	Liveness Finding	3
	Design Recommendation	3
6.2	EVM DSToken Divergences	3
	Differences Imposed by the Native Asset Architecture	4
	Differences Not Required by Stellar	4
6.3	Registry and SAC Authorization Consistency	5
6.4	Threat Model Review	5
6.5	Assessment of Design Decisions	6
	Freeze before seizure (§9.1)	6
	Wallet removal by every operator (§9.2)	6
	Roles surviving grantor revocation (§9.4)	6
	Ungoverned upgrades (§9.7)	6
6.6	OpenZeppelin Soroban 0.7.2	7
6.7	Test-Suite Assessment	7
6.8	Production Readiness Statement	8
7	Findings	10
7.1	Medium Risk	10
	7.1.1 A deleted Classic trustline blocks controller lock of every wallet for that investor	10
	7.1.2 Missing EVM seizure-destination restriction allows transfer operators to seize into self-designated custody	10
	7.1.3 remove/add wallet operations allow any registry operator to reassign funded wallets to another investor	11
7.2	Low Risk	13
	7.2.1 Missing EVM creator checks allow exchange operators to remove other operators' records	13
	7.2.2 Lower-privileged operators can block a MASTER-initiated admin transfer	13
	7.2.3 Selling-liability minimums prevent reliable full-balance burn and seizure	14
	7.2.4 Unbounded investor wallet counts can exceed atomic lock and unlock transaction limits	17
	7.2.5 Classic claimable balances fall outside controller burn and seizure	17
7.3	Informational	19
	7.3.1 Overlapping SAC error codes can misreport controller failures	19
	7.3.2 TTL renewal does not keep related controller state live	19

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize's Stellar DSToken protocol lets issuers represent securities as tokens on the Stellar network and manage them throughout their lifecycle. It links investor records to wallet addresses, allowing approved operators to onboard investors and manage their access to the asset. Identity verification and compliance decisions happen off-chain, while the protocol provides the on-chain controls to apply those decisions.

Authorized operators can issue tokens, restrict an investor's ability to transact, and burn or forcibly move tokens when needed for recovery or regulatory action. Different roles determine who can perform these actions. Investors transfer tokens using Stellar's native infrastructure, subject to the asset's authorization rules. The protocol adds administrative controls around those holdings rather than processing every transfer itself, and the asset issuer retains separate authority through its Stellar account.

5 Audit Scope

The audit scope was limited to:

```
contracts/asset-controller/src/access.rs
contracts/asset-controller/src/contract.rs
contracts/asset-controller/src/error.rs
contracts/asset-controller/src/events.rs
contracts/asset-controller/src/lib.rs
contracts/asset-controller/src/registry.rs
contracts/asset-controller/src/storage.rs
contracts/asset-controller/src/token.rs
contracts/asset-controller/src/types.rs
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Stellar DSToken](#) code provided by [Securitize](#). The findings consist of 3 Medium & 5 Low severity issues with the remainder being informational.

6.1 Access Control

Our examination supports the following conclusions across sequences of the audited controller's endpoints, starting from valid initialization and assuming the audited code is not replaced:

- **Role exclusivity.** Each address holds at most one role. Single and batch assignments reject existing membership, and admin acceptance rechecks the recipient's eligibility.
- **Declared grant hierarchy.** Grants and revocations follow the permissions below. Ordinary role management cannot grant or revoke MASTER.
- **Observable MASTER singularity.** Acceptance updates the OpenZeppelin admin, rechecks the recipient's eligibility, then grants the new MASTER membership and revokes the old one, all within one atomic transaction. No external call occurs between these state changes, and failure rolls back the entire transfer. The intermediate two-member state is not externally observable. After every successful public call, `get_role_member_count(MASTER) == 1`, and that member is the contract administrator.

Role	Token operations	Registry and investor locks	Grant/revoke	Pause, upgrade
MASTER	Issue, burn, seize	Yes	All operational roles	Yes
ISSUER	Issue, burn	Yes	ISSUER, EXCHANGE	No
TRANSFER	Burn, seize	Yes	TRANSFER	No
EXCHANGE	None	Yes	None	No

Only MASTER initiates admin succession; the nominated recipient must accept it. These conclusions concern the controller, not the Classic issuer's independent authority.

Liveness Finding

[L-2] "Lower-privileged operators can block a MASTER-initiated admin transfer". An ISSUER or TRANSFER operator can assign a conflicting role to the pending administrator, causing acceptance to fail with `RoleAlreadyAssigned`. This is a recoverable succession delay, not a failure of exclusivity or singularity. The current MASTER retains authority and can remove the conflicting role or cancel the transfer. Preventing conflicting assignments would avoid this interference.

Design Recommendation

Shared registry administration is intentional under §2.3, not a violation of the declared hierarchy. Nevertheless, wallet associations are critical to investor accounting and compliance. We recommend considering a dedicated registry role appointed and revoked by MASTER, rather than allowing every operator to remove wallets registered by other operators.

6.2 EVM DSToken Divergences

Native Stellar adaptations preserve authenticated administration and transaction atomicity, but change what the controller can freeze, recover and monitor. Their security acceptability depends on issuer custody, native recovery procedures and complete accounting. Other differences remove EVM safeguards.

Differences Imposed by the Native Asset Architecture

- **Transfers and freezing.** Native transfers do not consult the controller, so its pause is not an asset-wide halt. Investor locking preserves registration and calls `SAC set_authorized(false)`. This blocks ordinary receipt and spending but leaves existing Classic offers executable. Emergency containment is therefore weaker than an EVM transfer pause unless issuer-side hard freezing is available.
- **Burn and seizure.** Seizure atomically claws back and mints. SAC clawback cannot reduce the balance below outstanding selling liabilities, even when the nominal balance covers the request. A prior soft freeze through `set_authorized(false)` leaves existing offers and their liabilities intact. [L-3] “Selling-liability minimums prevent reliable full-balance burn and seizure” identifies the additional issuer-side hard-freeze and liability-clearing dependency. This can delay recovery despite correct operator authorization; historical clawback eligibility and destination capacity must also be verified.
- **Independent issuer and claimable balances.** The issuer retains native powers outside controller governance. [L-5] “Classic claimable balances fall outside controller burn and seizure” concerns a different recovery limit. These tokens occupy separate ledger entries, not the originating trustline. Eligible entries require issuer-authorized whole-entry destruction, with separate replacement issuance where needed. Both this and liability clearing require usable issuer authority, but they are distinct operations. Neither controller pause nor wallet deauthorization completes recovery.
- **Holdings and units.** Classic trustlines, contract balances and muxed payment identifiers have different custody semantics. Fixed seven-decimal units, Classic balance limits and the absence of transparent rebasing require migration checks. Incorrect scaling or treating muxed identifiers as separate holdings can corrupt accounting or misdirect recovery. These differences are acceptable only with explicit conversion and custody rules.
- **Holder permissions and indexing.** SAC permits holder and allowance-funded burning; allowances expire by ledger deadline. Native activity can occur without controller events, and the fee payer need not be the authorizing operator. Controller events alone cannot establish supply or operator attribution. Clients must disclose delegated destruction and reconcile native activity without double-counting seizure.
- **State lifetime and resources.** Soroban records require renewal or restoration, and native locking touches each wallet. The additional risk is administrative unavailability, not silent deletion or revocation. Restoration-aware transaction preparation is required; accepting more wallets than the lock API can process is separately avoidable.

Differences Not Required by Stellar

- **Registry protections.** [M-3] “remove/add wallet operations allow any registry operator to reassign funded wallets to another investor” concerns missing EVM zero-balance checks. Reassignment can misdirect investor benefits without moving tokens or obtaining holder authorization. Removing a funded wallet and adding it to an unlocked investor can also revive spending under a retained, unexpired SAC allowance. EVM rejects that funded removal. Restore the balance checks across onboarding and removal. [L-1] “Missing EVM creator checks allow exchange operators to remove other operators’ records” separately identifies weakened operator isolation. Investor locking preserves registration without requiring either omission.
- **Seizure custody.** [M-2] “Missing EVM seizure-destination restriction allows transfer operators to seize into self-designated custody” concerns lost independent approval. Unlike EVM, Stellar accepts a destination registered by the executing transfer agent. A compromised agent can therefore redirect holdings into its own custody. Destination approval should remain separate from investor registration.
- **Lock authority and availability.** Every Stellar operator can freeze investors and, when unpaused, unlock them, expanding the impact of compromise beyond EVM’s transfer-agent/master permissions. [M-1] “A deleted Classic trustline blocks controller lock of every wallet for that investor” and [L-4] “Unbounded investor wallet counts can exceed atomic lock and unlock transaction limits” identify separate enforcement delays. Handle verified absent trustlines, bound enrollment and explicitly approve the broader lock authority.
- **Pause policy.** Stellar reserves pause to MASTER and blocks issuance/burn/seizure and investor unlock, while wallet removal and investor freeze remain available. EVM permits transfer-agent pause and leaves

direct issue/burn/seize available. Stellar pause neither stops every abusive registry operation nor preserves immediate controller confiscation. The emergency procedure must explicitly account for this tradeoff.

- **Reduced business interfaces.** Caps, custody classes, metadata, issuance schedules and accounting views are omitted or simplified. These become security gaps where the asset requires the missing policy; otherwise they are integration differences. Approve exclusions explicitly, adapt identifiers and batch inputs, and do not treat controller-only counters as asset-wide supply.
- **Governance and maintenance.** [L-2] “Lower-privileged operators can block a MASTER-initiated admin transfer” describes recoverable succession interference, not loss of singularity. The Informational finding [I-2] “TTL renewal does not keep related controller state live” concerns restoration and maintenance, not permanent state loss. [I-1] “Overlapping SAC error codes can misreport controller failures” concerns diagnosis. These require succession safeguards, restoration-aware tooling and origin-aware error decoding. Missing migration and upgrade-notification interfaces also require operational planning.

Issuer burn-plus-mint authority, same-address seizure, operator-less token events and empty bulk-burn reasons are inherited EVM behavior, not new parity failures. Controls belonging only to other compliance variants are not assumed active in the Whitelisted baseline.

6.3 Registry and SAC Authorization Consistency

Taking investor locking into account, the invariant holds after successful controller registry and lock updates:

Controller state	SAC authorization
Registered, investor unlocked	Authorized
Registered, investor locked	Deauthorized (<code>set_authorized(false)</code>)
Successfully removed	Unregistered and deauthorized

A registered, locked wallet is expected. `attach_wallet`, `remove_wallet` and `lock_investor` write both sides in one transaction; a SAC failure rolls back both. New wallets inherit the investor’s lock flag.

The Classic issuer can still call `SetTrustLineFlags` and deauthorize a wallet that remains registered. That would break the table above, but it is an explicit action by the protocol owner, not a controller defect.

6.4 Threat Model Review

The operator-compromise scenarios are drawn correctly. Freeze language in the request predates `lock_investor`. That text is stale, not wrong, and the evaluation below reads it with locking in place.

Stale (written before investor locking)

- Scenarios 6 and 7 freeze via `remove_wallet` and treat a registered, deauthorized wallet as a consistency bug. With `lock_investor`, the wallet stays registered and burn and seize still work. Registered and locked is expected (§6.3). Removal is a different operation and still blocks recovery.

Missed

- Any registry operator can reassign a funded wallet to another investor and revive a still-valid SAC allowance ([M-3]).
- `TRANSFER` can register a seizure destination and seize into it ([M-2]).
- Every operator, including `EXCHANGE`, can unlock while unpaused. A legitimate investor-wide freeze can still fail on a deleted trustline ([M-1]) or an oversized wallet set ([L-4]).
- Holders can open DEX offers and claimable balances. Controller burn and seize cannot recover those tokens without issuer-side procedures ([L-3], [L-5]).

6.5 Assessment of Design Decisions

Freeze before seizure (§9.1)

Unsafe. The removal-then-seize procedure deletes the wallet's registration and therefore prevents controller seizure, while also allowing the wallet to be linked to a different investor. Although `lock_investor` now offers a replacement that preserves registration and permits clawback without reauthorizing the source, `remove_wallet` remains available, so the unsafe sequence is still callable. We recommend replacing the removal-based procedure with investor locking and rejecting wallet addition and removal when the wallet has a positive balance.

The replacement lock-then-seize procedure is **safe only under the condition that the controller is unpaused for seizure and the holdings satisfy Stellar's native recovery requirements**, including clawback eligibility and an authorized destination with sufficient capacity. Since locking neither clears selling liabilities nor recovers claimable balances, we recommend maintaining issuer-side recovery procedures for those cases ([L-3], [L-5]); these are conditions on recovery, rather than reasons to regard the registration-preserving lock itself as unsafe.

Wallet removal by every operator (§9.2)

Unsafe. Every operator can remove wallets administered by other operators, including those with greater privileges, which weakens the separation of responsibilities between roles ([L-1]). Removing funded wallets also permits their reassignment to different investors ([M-3]), although the broader concern is that onboarding access carries authority to dismantle other operators' registrations.

We recommend limiting wallet addition and removal to a designated subset of roles, with ownership or approval rules that prevent lower-privilege operators from overriding higher-privilege administration, and rejecting both operations when the wallet has a positive balance. We also recommend pause-gating removal as an additional containment measure, rather than as a substitute for restricting authority, because an `EXCHANGE` with the same permissions could still cause this disruption when unpaused. Since `lock_investor` already separates freezing from deregistration, we recommend using that distinction in operational procedures and assigning freeze and unlock permissions according to compliance responsibility rather than onboarding access.

Roles surviving grantor revocation (§9.4)

Safe only if containment includes every suspect delegate and verifies remaining roles before unpausing. Pausing blocks new grants but not `remove_role`, so surviving `ISSUER` accounts can still revoke peers during cleanup. We recommend enumerating members through `get_role_member` and reconciling membership changes against a trusted inventory, or revoking all operational roles and rebuilding the approved set if trust cannot be established.

Even with this recovery path, allowing `ISSUER` and `TRANSFER` operators to appoint an unbounded number of peers is difficult to justify without a clear business requirement, particularly because those delegates can appoint further peers and substantially increase the time needed to contain an incident. We therefore recommend reserving role assignment to `MASTER` or a dedicated role administrator, rather than allowing operational roles to replicate their own authority without a bound.

Ungoverned upgrades (§9.7)

Unsafe when a single `MASTER` signing key is the only approval, because compromise of that key permits replacement of every controller safeguard. We recommend independent threshold approval together with a review delay as minimum upgrade controls, using an arrangement such as 5-of-7 or 7-of-9 where signer independence, key custody and recovery procedures support that threshold, and preserving those controls through admin succession.

We recommend a delay between upgrade approval and execution that gives participants time to review changes to core protocol functionality, while recognizing that notice alone does not guarantee an exit. For an `AUTH_REQUIRED` asset, holders can transfer only to authorized wallets, and moving tokens to another wallet does not avoid an asset-wide change, so we recommend identifying an available redemption or other supported exit mechanism before presenting the delay as an opportunity to withdraw.

The conditional verdicts require demonstrated production procedures. They do not confirm that those procedures are already in place.

6.6 OpenZeppelin Soroban 0.7.2

OpenZeppelin's 0.7.0 release candidate underwent a [security audit](#) covering access control, role transfer, pausing and upgradeability, while the subsequent 0.7.2 patch updated the Soroban SDK.

Our review of public sources identified no outstanding published vulnerabilities affecting the access-control and upgradeability modules used by this controller. Although the library remains designated as [experimental](#), 0.7.2 is currently the latest non-prerelease in the audited 0.7 release line. We recommend retaining the exact version pins, monitoring upstream security disclosures and reviewing dependency changes before adoption.

6.7 Test-Suite Assessment

Happy-path coverage is complete at the endpoint level. The 64 passing controller tests exercise successful calls to all 32 public methods, with successful construction covered separately. This includes getters and inherited pause/upgrade endpoints; it does not mean every role, input boundary or operation sequence is covered. No public method lacks a basic successful-call test.

The principal findings relate to missing security assertions or operational scenarios, rather than missing happy-path calls:

Finding	Could testing have identified it?
[M-1] — Deleted trustline blocks investor locking	Already exposed by a unit test. <code>sac_failure_rolls_back_earlier_wallet_changes_lock_state_and_events</code> asserts that a missing trustline causes the entire lock to roll back. It establishes atomicity, but treats the inability to freeze the remaining wallets as expected behavior.
[M-2] — Self-designated seizure custody	Yes, with a unit test asserting that registration by the executing transfer operator is insufficient custody approval. The existing successful seizure test accepts such a destination, so it validates the implementation rather than the EVM restriction.
[M-3] — Funded-wallet reassignment	Yes, with a unit test asserting that a funded wallet cannot be detached and associated with another investor. The existing removal/re-registration test returns the wallet to the same investor and accepts funded removal, so it does not enforce attribution protection.
[L-1] — Cross-operator record removal	Yes, with a unit test checking that an exchange cannot remove records administered by another operator under the intended isolation policy. General role-membership checks do not establish record-level authority.
[L-2] — Role grants block admin acceptance	Already exposed by a unit test. <code>admin_transfer_rechecks_exclusive_role_and_rolls_back_on_conflict</code> checks rejection and recovery from a conflicting role. It protects role exclusivity without preventing succession interference.
[L-3] — Selling liabilities constrain recovery	Yes, through a Classic-ledger integration test checking recovery of the full balance when outstanding selling liabilities exist. The current amount/trustline-limit tests do not cover this separate recovery constraint.
[L-4] — Wallet counts exceed atomic lock limits	Yes, through a resource-boundary test comparing the largest supported investor wallet set with deployment-network transaction limits. Small-list completeness and rollback tests do not establish that every permitted enrollment remains lockable.

Finding	Could testing have identified it?
[L-5] — Claimable balances require issuer recovery	Yes, through an integration test distinguishing tokens on a registered trustline from tokens held in claimable-balance entries. Ordinary wallet burn/seizure tests do not establish recovery of the latter.

We recommend adding the missing security-policy tests alongside accepted remediations and updating existing failure-characterization tests where the intended behavior changes. Fuzzing could broaden sequence coverage, but is not necessary to identify the specific gaps above. The six deployment-script unit tests also passed; network integration scenarios were not executed in this assessment.

6.8 Production Readiness Statement

Before mainnet release, we recommend addressing three areas:

1. **Remediation of the identified issues.** We recommend prioritizing the medium-severity findings and the low-severity findings affecting recovery and administrative availability, with the implemented mitigations reviewed before release. For limitations inherent to Stellar, we recommend documented recovery procedures and explicit acceptance of any remaining risk.
2. **Review and tightening of the access-control model.** We recommend clearer separation of registry administration, compliance actions and custody approval, together with restrictions on peer role assignment so that operators cannot create an unbounded set of delegates. Independent upgrade approval and a review delay would complement these changes.
3. **Operational security for MASTER and the Classic issuer.** Although outside the scope of the codebase review, custody and availability of these authorities are critical to the design. The Classic issuer needs to remain an active, readily available participant in recovery involving hard freezes, selling liabilities and claimable balances, rather than being treated as a deployment-only account. This places particularly high demands on procedures that are both secure and practical to execute promptly. We recommend threshold authorization with independent signers and separately secured devices for both authorities, supported by protected backups, transaction review and rehearsed recovery procedures. Compromise of either authority could undermine the system's safeguards, while issuer unavailability could prevent required recovery even when the controller behaves correctly.

Summary

Project Name	Stellar DSToken
Repository	bc-stellar-dstoken-sc
Commit	23c0885b11fa...
Fix Commit	b7ea84de7735...
Audit Timeline	Sep 15th - Sep 21st, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	3
Low Risk	5
Informational	2
Gas Optimizations	0
Total Issues	10

Summary of Findings

[M-1] A deleted Classic trustline blocks controller lock of every wallet for that investor	Resolved
[M-2] Missing EVM seizure-destination restriction allows transfer operators to seize into self-designated custody	Resolved
[M-3] remove/add wallet operations allow any registry operator to reassign funded wallets to another investor	Resolved
[L-1] Missing EVM creator checks allow exchange operators to remove other operators' records	Resolved
[L-2] Lower-privileged operators can block a MASTER-initiated admin transfer	Resolved
[L-3] Selling-liability minimums prevent reliable full-balance burn and seizure	Acknowledged
[L-4] Unbounded investor wallet counts can exceed atomic lock and unlock transaction limits	Resolved
[L-5] Classic claimable balances fall outside controller burn and seizure	Acknowledged
[I-1] Overlapping SAC error codes can misreport controller failures	Resolved
[I-2] TTL renewal does not keep related controller state live	Acknowledged

7 Findings

7.1 Medium Risk

7.1.1 A deleted Classic trustline blocks controller lock of every wallet for that investor

Description: An investor can block their wallets from being locked by deleting the asset trustline from one empty wallet. Its stale registration causes both `lock_investor` and `remove_wallet` to revert, preventing the controller from freezing the investor or removing the blocking wallet.

`lock_investor_after_auth` requires every associated wallet to be included, then updates each wallet's SAC authorization:

```
for wallet in wallets.iter() {
    sac.set_authorized(&wallet, &!locked);
}
investor.locked = locked;
storage::set_investor(env, investor_id, &investor);
```

For a Classic wallet with a deleted trustline, `set_authorized` fails with `TrustlineMissingError`. The entire lock rolls back, including any earlier deauthorizations, so the investor's other wallets remain authorized. The completeness check prevents omitting the affected wallet.

`remove_wallet` fails for the same reason: it attempts deauthorization before deleting the stale association.

```
StellarAssetClient::new(&env, &storage::sac(&env)).set_authorized(&wallet, &false);
storage::remove_wallet(&env, &wallet);
storage::set_investor(&env, &investor_id, &investor);
```

Without the holder recreating the trustline, operators must work around the failed investor-wide lock by freezing the remaining live trustlines through the Classic issuer.

Impact: FR-7 requires authorization revocation for sanctions, revoked KYC, and court orders—time-sensitive enforcement actions. An investor can prepare an empty wallet and delete its trustline before enforcement, causing the intended freeze to fail while their funded wallets remain usable.

The additional issuer-side coordination can give the investor meaningful time to sell or transfer tokens before the protocol stops them. The impact is delayed enforcement across the investor's wallets, not permanent loss of issuer control.

Recommended Mitigation: In `lock_investor` when freezing, and in `remove_wallet`, skip `set_authorized(false)` if the Classic trustline is verifiably absent.

Securitize: Fixed in commit [df917](#)

Cyfrin: Verified.

As an informational note, the `Err(Err(invoker_error))` in the code handling of `try_set_authorized` is unreachable since the SAC client uses the generic `soroban_sdk::Error` and conversion into that same error type cannot fail.

7.1.2 Missing EVM seizure-destination restriction allows transfer operators to seize into self-designated custody

Description: The Stellar implementation removes a separation-of-duties control present in the EVM implementation: a transfer agent may execute a seizure, but the custody destination must be independently approved by an issuer or master.

On EVM, `ComplianceService::validateSeize` rejects any destination that is not an issuer-special wallet:

```
require(getWalletManager().isIssuerSpecialWallet(_to), "Target wallet type error");
```

Designating a destination through `WalletManager::addIssuerWallet` requires `onlyIssuerOrAbove`. That guard explicitly permits only `ISSUER` and `MASTER`, excluding `TRANSFER_AGENT`:

```
function addIssuerWallet(address _wallet) public override onlyIssuerOrAbove returns (bool) {
    return setSpecialWallet(_wallet, ISSUER);
}

modifier onlyIssuerOrAbove {
    IDSTrustService trustManager = getTrustService();
    uint8 role = trustManager.getRole(msg.sender);
    require(role == ROLE_ISSUER || role == ROLE_MASTER, "Insufficient trust level");
    _;
}
```

The transfer agent can therefore choose among approved custody destinations, but cannot register an arbitrary wallet and make it eligible for seizure.

In contrast, in the Stellar `AssetController::seize` function, any registered wallet is accepted as the seizure destination.

```
if storage::wallet_investor_and_extend_ttl(&env, &from).is_none()
|| storage::wallet_investor_and_extend_ttl(&env, &to).is_none()
{
    panic_with_error!(&env, AssetControllerError::WalletNotRegistered);
}

let sac = StellarAssetClient::new(&env, &storage::sac(&env));
sac.clawback(&from, &amount);
sac.mint(&to, &amount);
```

Since `TRANSFER` can also call `AssetController::register_investor` and `AssetController::add_wallet`, it can register an attacker-controlled wallet and then seize investor holdings into it without separate `ISSUER` or `MASTER` approval.

Section 2.5 of `stellar-audit-request.md` identifies parity with the EVM implementation as an acceptance criterion. The destination-approval restriction can also be enforced on Stellar, so its omission is an unnecessary deviation that expands the transfer operator's authority.

Impact: A malicious or compromised `TRANSFER` operator can redirect investor holdings into a wallet it controls, rather than being limited to custody destinations approved by `ISSUER` or `MASTER` as in the EVM model.

Administrative clawback does not replace this preventive control: recovery requires intervention, and the attacker may move or burn the received tokens before that occurs.

Recommended Mitigation: Restore the EVM separation of authority by maintaining seizure-destination approval independently of investor registration. Restrict this approval to `MASTER` or `ISSUER`, and require it in `AssetController::seize` before clawback and mint. Registering a wallet must not automatically approve it as a seizure destination.

Securitize: Fixed in commit [8ea79](#)

Cyfrin: Verified.

7.1.3 remove/add wallet operations allow any registry operator to reassign funded wallets to another investor

Description: `AssetController::remove_wallet` and `AssetController::add_wallet` do not require the wallet's token balance to be zero. This allows registry operators to reassign a funded wallet to a different investor without moving its tokens or obtaining holder authorization.

In the EVM implementation, both registry operations require a zero balance, preventing reassignment of funded wallets.

```
require(getToken().balanceOf(_address) == 0, "Wallet with positive balance");
```

Stellar's `AssetController::remove_wallet` deauthorizes the wallet and deletes its investor association without checking its token balance.

```
StellarAssetClient::new(&env, &storage::sac(&env)).set_authorized(&wallet, &false);  
storage::remove_wallet(&env, &wallet);  
storage::set_investor(&env, &investor_id, &investor);
```

Although the tokens remain in the wallet, `AssetController::add_wallet` checks only that the wallet is currently unregistered and the destination investor exists. It does not check the wallet's balance or previous investor association. The registry's `attach_wallet` helper applies the destination investor's lock policy and stores the new association:

```
sac.set_authorized(&wallet, &!investor.locked);  
  
storage::set_wallet_investor(env, &wallet, &investor_id);  
storage::set_investor(env, &investor_id, investor);
```

Both operations permit MASTER, ISSUER, TRANSFER, and EXCHANGE callers without holder authorization. Consequently, an authorized registry operator can remove and reattach a funded wallet under a different investor without moving its tokens.

Impact: A compromised operational key, including an EXCHANGE key, can reattribute multiple funded wallets to a colluding investor without moving tokens or obtaining their holders' signatures. Systems that rely on these associations for ownership accounting, distributions, redemption, voting, or compliance may recognize the wrong investor and misdirect benefits.

Recommended Mitigation: Require a zero token balance before removing or attaching a wallet, consistently across registration and bulk onboarding paths. Use investor lock/unlock functionality for funded wallets that require authorization changes without changing their investor association.

Securitize: Fixed in commit [65ec3](#)

Cyfrin: Verified.

7.2 Low Risk

7.2.1 Missing EVM creator checks allow exchange operators to remove other operators' records

Description: The Stellar implementation omits the EVM restriction that limits EXCHANGE operators to only remove records that they themselves created.

In the EVM implementation, `RegistryService::removeWallet` checks the wallet record's creator:

```
require(getTrustService().getRole(msg.sender) != EXCHANGE || investorsWallets[_address].creator ==  
↳ msg.sender, "Insufficient permissions");
```

`RegistryService::removeInvestor` applies the same restriction to the investor record:

```
require(getTrustService().getRole(msg.sender) != EXCHANGE || investors[_id].creator == msg.sender,  
↳ "Insufficient permissions");
```

These checks prevent one exchange from removing another operator's records without restricting the broader removal authority of MASTER, ISSUER, and TRANSFER_AGENT.

On Stellar, `AssetController::remove_wallet` and `AssetController::remove_investor` permit EXCHANGE callers but do not check who created the record. Wallet removal deauthorizes the wallet and deletes its association:

```
StellarAssetClient::new(&env, &storage::sac(&env)).set_authorized(&wallet, &false);  
storage::remove_wallet(&env, &wallet);  
storage::set_investor(&env, &investor_id, &investor);
```

Consequently, Exchange B can remove wallets registered by Exchange A or by MASTER, ISSUER, or TRANSFER operators. `AssetController::remove_wallet` remains available while paused. Once an investor has no associated wallets, Exchange B can also remove that investor regardless of its creator, but `AssetController::remove_investor` requires the controller to be unpaused.

Section 2.5 of `stellar-audit-request.md` identifies EVM parity as an acceptance criterion. Stellar does not prevent storing record provenance or checking it during removal; omitting these restrictions is an expansion of exchange authority.

Impact: A malicious or compromised exchange operator can disrupt another operator's investor registrations and deauthorize its registered wallets. Restoring those associations requires administrative intervention.

Recommended Mitigation: Store the creating operator separately for each investor and wallet record. When an EXCHANGE caller removes a record, require that caller to match the corresponding creator, preserving the EVM permissions for the other operational roles.

Securitize: Fixed in commits [65ec3](#) and [c2471](#).

Cyfrin: Verified.

7.2.2 Lower-privileged operators can block a MASTER-initiated admin transfer

Description: `AssetController::transfer_contract_admin` requires the proposed admin to have no role, and `access::accept_contract_admin` repeats that check before granting MASTER:

```
require_available_admin(env, &new_admin);  
grant_role_with_event(env, &new_admin, &MASTER_ROLE, &new_admin);  
revoke_role_with_event(env, &previous_admin, &MASTER_ROLE, &new_admin);
```

The availability check rejects any existing membership:

```
if get_role(env, account) != NONE_ROLE {  
    panic_with_error!(env, AssetControllerError::RoleAlreadyAssigned);  
}
```

However, ordinary role assignment does not exclude the pending recipient. An ISSUER can grant it EXCHANGE, or a TRANSFER can grant it TRANSFER, without the recipient's consent. Acceptance then fails with `RoleAlreadyAssigned`, allowing either operator to interfere with a transfer initiated by MASTER.

Impact: A malicious or compromised issuer or transfer operator can delay admin succession and repeat the interference while role assignment remains available. The old master and pending proposal remain intact, so this does not permit takeover or permanent lockout. Recovery may require pausing the controller, removing the conflicting role, and completing acceptance while paused.

Recommended Mitigation: Consider preventing ordinary role assignments to the recipient of an active admin-transfer proposal. Retain the acceptance-time exclusivity check so the new admin cannot acquire MASTER alongside an operational role.

Securitize: Fixed in commit [2522431](#).

Cyfrin: Verified.

7.2.3 Selling-liability minimums prevent reliable full-balance burn and seizure

Description: The controller's administrative `burn` and `seize` functions cannot be relied upon to recover a Classic account's full holding under normal operating conditions. Both use Stellar Asset Contract (SAC) clawback, which cannot reduce a G-address trustline balance below its outstanding selling liabilities. Consequently, **any positive selling liability prevents a full-balance burn or seizure**, even when the operator is authorized, the wallet is registered and clawback-enabled, and its nominal balance covers the requested amount.

The protocol controls the Classic issuer and can clear liabilities through a hard freeze, so the obstruction is not inherently unresolvable.

- Native clawback path

`AssetController::burn` reaches `sac.clawback` through `token::burn_after_auth`. `AssetController::seize` uses the same debit before minting to the destination:

```
let sac = StellarAssetClient::new(&env, &storage::sac(&env));
sac.clawback(&from, &amount);
sac.mint(&to, &amount);
```

`StellarAssetClient` is only the SDK invocation interface. The actual implementation is built into the Soroban host. For a non-issuer G-address, the issued-asset balance is stored in its Classic trustline, so clawback must update that trustline rather than a separate contract balance:

```
AssetController::burn / AssetController::seize
-> native SAC::clawback
-> spend_balance_no_authorization_check
-> transfer_classic_balance(account, -clawback_amount)
-> transfer_trustline_balance
-> get_min_max_trustline_balance
-> reject if the resulting balance is below selling liabilities
```

The native `clawback implementation` authenticates the SAC administrator and then invokes the debit helper:

```
pub(crate) fn clawback(e: &Host, from: Address, amount: i128) -> Result<(), HostError> {
    let _span = tracy_span!("SAC clawback");
    check_nonnegative_amount(e, amount)?;
    check_clawbackable(e, from.metered_clone(e)?);
    check_not_issuer(e, &from)?;

    let admin = read_administrator(e)?;
    admin.require_auth()?;

    e.extend_current_contract_instance_and_code_ttl(
        INSTANCE_TTL_THRESHOLD.into(),
        INSTANCE_EXTEND_AMOUNT.into(),
    )
}
```

```

    )?;

    spend_balance_no_authorization_check(e, from.metered_clone(e)?, amount)?;
    event::clawback(e, from, amount)?;
    Ok(())
}

```

The call to `spend_balance_no_authorization_check` above reaches the following [G-address branch](#). It converts the requested amount to `i64` and explicitly calls `transfer_classic_balance` with a negative amount:

```

ScAddress::Account(acc_id) => {
    let i64_amount = i64::try_from(amount).map_err(|_| {
        e.error(
            ContractError::OverflowError.into(),
            "spent amount is too large for an i64",
            &[],
        )
    })?;
    transfer_classic_balance(e, acc_id, -i64_amount, &addr)
}

```

`transfer_classic_balance` then selects the asset type. Both issued-asset branches call the local `transfer_trustline_balance_unless_issuer` closure, which forwards a non-issuer account's debit to `transfer_trustline_balance`:

```

pub(crate) fn transfer_classic_balance(
    e: &Host,
    to_key: AccountId,
    amount: i64,
    addr: &Address,
) -> Result<(), HostError> {
    let transfer_trustline_balance_unless_issuer =
        |asset: TrustLineAsset, issuer: AccountId, to: AccountId| -> Result<(), HostError> {
            if issuer == to {
                return Ok(());
            }

            transfer_trustline_balance(e, to, asset, amount)
        };

    match read_asset(e)? {
        Asset::Native => transfer_account_balance(e, to_key, amount, addr),
        Asset::CreditAlphanum4(asset) => {
            let issuer = asset.issuer.metered_clone(e)?;
            let tlasset = TrustLineAsset::CreditAlphanum4(asset);
            transfer_trustline_balance_unless_issuer(tlasset, issuer, to_key)
        }
        Asset::CreditAlphanum12(asset) => {
            let issuer = asset.issuer.metered_clone(e)?;
            let tlasset = TrustLineAsset::CreditAlphanum12(asset);
            transfer_trustline_balance_unless_issuer(tlasset, issuer, to_key)
        }
    }
}

```

This is the same clawback debit throughout, not a separate holder transfer. Skipping the holder's spending-authorization check does **not** skip the trustline balance constraints reached below.

- The selling-liability minimum

`transfer_trustline_balance` obtains the permitted balance range and rejects a debit outside it:

```

let (min_balance, max_balance) = get_min_max_trustline_balance(host, &tl)?;

```

```

let Some(new_balance) = t1.balance.checked_add(amount) else {
    return Err(host.error(
        ContractError::BalanceError.into(),
        "resulting balance overflow",
        &[],
    ));
};
if new_balance >= min_balance && new_balance <= max_balance {
    t1.balance = new_balance;
    le = Host::modify_ledger_entry_data(host, &le, LedgerEntryData::Trustline(t1))?;
    storage.put(&lk, &le, None, &host, None)
} else {
    Err(err!(
        host,
        ContractError::BalanceError,
        "resulting balance is not within the allowed range",
        min_balance,
        new_balance,
        max_balance
    ))
}
}

```

The pivotal check is in `get_min_max_trustline_balance`. For a trustline with liability data, the function explicitly sets the minimum balance to its selling liabilities:

```

if let TrustLineEntryExt::V1(ext1) = &t1.ext {
    let min_balance = ext1.liabilities.selling;
    if t1.limit < ext1.liabilities.buying {
        return Err(e.err(
            ScErrorType::Storage,
            ScErrorCode::InternalError,
            "limit is lower than liabilities",
            &[],
        ));
    }
    let max_balance = t1.limit - ext1.liabilities.buying;
    Ok((min_balance, max_balance))
} else {
    let min_balance = 0;
    let max_balance = t1.limit;
    Ok((min_balance, max_balance))
}
}

```

For example, a holder with balance **100** and selling liabilities of only **1** cannot have all **100** burned or seized: the resulting balance would be zero, below the required minimum of one. If all 100 are reserved by offers, even a debit of one fails. Thus, ordinary offer placement can prevent complete recovery; the holder need not reserve its entire balance.

- Soft deauthorization does not resolve the obstruction

SAC `set_authorized(false)` preserves existing offers and liabilities by setting the maintain-liabilities flag:

```

t1.flags &= !(TrustLineFlags::AuthorizedFlag as u32);
t1.flags |= TrustLineFlags::AuthorizedToMaintainLiabilitiesFlag as u32;

```

Clearing the obstruction instead requires an issuer-authorized Classic hard freeze that clears **both** authorization flags. `remove_wallet` is not a substitute: it uses soft deauthorization and also deletes the registration required by controller burn/seizure. The recovery flow must preserve registration and keep the source frozen while liabilities are cleared and the controller operation is retried.

Impact: Ordinary selling liabilities can prevent full-position burn or seizure, making a core recovery function dependent on additional issuer intervention. This may impose significant coordination and delays, particularly if issuer

authority is separately governed.

Recommended Mitigation: Implement and document an operating flow for recovering holdings from every supported investor wallet with outstanding liabilities. The flow must coordinate issuer-side hard freeze and liability clearing, preserve controller registration and clawback eligibility, then execute burn or seizure without reauthorizing the source.

Securitize: Acknowledged. Documentation was expanded to clarify the operating procedures for the issuer path in commit [0cb8e](#).

Cyfrin: Acknowledged.

7.2.4 Unbounded investor wallet counts can exceed atomic lock and unlock transaction limits

Description: `AssetController::lock_investor` requires every wallet associated with an investor to be processed in one transaction. However, `add_wallet_after_auth` does not enforce a transaction-safe wallet-count limit. Repeated, individually valid onboarding calls can therefore create an investor whose complete wallet set exceeds transaction resource limits.

Splitting the list across successive calls is not supported: each invocation requires `wallets.len() == investor.wallet_count`, with every wallet unique and belonging to that investor. A resource-limit failure rolls back the entire invocation without changing the lock policy or wallet authorizations.

The queried mainnet configuration on **2026-09-16** differed from the SDK's bundled test preset:

Per-transaction resource	Mainnet	SDK 26.1.0 test preset
Footprint entries	400	100
Read-write entries	200	50
Event bytes	16,384	16,384

If an investor has 83 wallets, the total event bytes is 16,400, exceeded the cap and causing a revert.

Impact: An unusually large investor wallet set can prevent normal investor-wide locking or unlocking. Creating this condition requires authorized onboarding, whether through legitimate enrollment, operator error, or collusion with an operator.

Operators can recover by removing and deauthorizing wallets individually until the remaining set fits. However, removed wallets lose their registration and eligibility for registration-dependent controller recovery.

Recommended Mitigation: Enforce a conservative per-investor wallet-count limit.

Securitize: Fixed in commit [5cb65](#)

Cyfrin: Verified.

7.2.5 Classic claimable balances fall outside controller burn and seizure

Description: `AssetController::burn` and `AssetController::seize` recover tokens from wallet balances through SAC clawback. Neither function can recover tokens held in native Stellar claimable balances, leaving an issuer-side dependency in the administrative recovery workflow.

A native Stellar claimable balance is a separate ledger entry, not a wallet or trustline. For a non-issuer holder, it inherits the source trustline's clawback-enabled status when [created](#).

Both controller recovery functions use address-based clawback. `burn` delegates to `token::burn_after_auth`, while `seize` claws back before minting to its destination:

```
let sac = StellarAssetClient::new(&env, &storage::sac(&env));
sac.clawback(&from, &amount);
```

```
sac.mint(&to, &amount);
```

Once tokens are held in a claimable entry, they are no longer part of the originating wallet's balance. Controller burn or seizure against that wallet therefore cannot recover them; insufficient-balance failures are correct at the SAC level but leave the intended recovery incomplete.

For entries created from clawback-enabled trustlines, recovery remains available through native [Clawback-ClaimableBalance](#). This operation requires authorization from the Classic asset issuer under its signer configuration; controller roles and SAC administrator status alone are insufficient. It destroys the **entire entry** and does not support partial recovery or transfer to a seizure destination.

Impact: Controller-only recovery cannot guarantee complete destruction or confiscation of affected holdings. Recovery additionally depends on identifying relevant claimable balances and obtaining Classic issuer authorization.

Recommended Mitigation: Maintain an issuer-authorized procedure to identify and claw back eligible claimable balances before declaring recovery complete or restoring wallet authorization.

Securitize: Acknowledged. Documentation was expanded to clarify the operating procedures for the issuer path in commit [0cb8e](#).

Cyfrin: Acknowledged.

7.3 Informational

7.3.1 Overlapping SAC error codes can misreport controller failures

Description: The controller propagates native Stellar Asset Contract (SAC) errors without translating them. Some native error codes overlap with `AssetControllerError` values that describe different failure conditions:

Code	Controller interpretation	Actual native failure
10	<code>WalletAlreadyRegistered</code>	<code>BalanceError</code>
11	<code>WalletNotRegistered</code>	<code>BalanceDeauthorizedError</code>
13	<code>WalletCountOverflow</code>	<code>TrustlineMissingError</code>

Insufficient balance reported as duplicate registration. In `AssetController::burn`, a positive amount and a registered source pass controller validation. If the requested clawback exceeds the source's available balance, the SAC rejects it with `BalanceError`, code 10. A client decoding this through `AssetControllerError` reports `WalletAlreadyRegistered`, incorrectly suggesting a duplicate-registration problem instead of insufficient balance.

Deauthorization reported as missing registration. In `AssetController::issue_tokens`, a registered wallet passes the registration check. If its native trustline has been independently deauthorized, SAC mint rejects it with `BalanceDeauthorizedError`, code 11. Decoding this as `WalletNotRegistered` incorrectly reports that the association is missing, when the actual problem is native authorization.

Missing trustline reported as wallet-count overflow. In `AssetController::add_wallet`, an existing investor and an unregistered G-address can pass the registry checks and checked count increment. If that address lacks the asset's trustline, the subsequent SAC authorization call fails with `TrustlineMissingError`, code 13. Decoding this as `WalletCountOverflow` incorrectly reports that the investor's wallet counter overflowed, even though the count check passed and the actual problem is the missing trustline.

Impact: Clients interpreting these native failures as controller errors can report an incorrect cause and direct operators toward unrelated recovery actions, such as registering an already-registered wallet or investigating a counter overflow when a trustline is missing. No authorization bypass or partial state change results from the ambiguity.

Recommended Mitigation: Translate these native failures into distinct, appropriately named controller errors at SAC call boundaries. Preserve existing public error codes and update client decoding for the translated errors.

Securitize: Fixed in commit [f34cc](#)

Cyfrin: Verified.

7.3.2 TTL renewal does not keep related controller state live

Description: Controller instance/code, investor records, and wallet mappings have independent lifetimes. Operations renew only selected entries, so activity involving an investor does not necessarily keep all related state live.

Operation	Instance/code	Investor record	Wallet mappings
Extend TTL	Renewed	Not renewed	Not renewed
Register investor / add wallet	Renewed	Renewed	New mapping only; existing siblings untouched
Remove wallet	Renewed	Renewed	Selected mapping deleted; siblings untouched
Issue / burn / seize, including batches	Renewed	Not renewed	Touched mappings renewed

Operation	Instance/code	Investor record	Wallet mappings
Combined registration and issuance	Renewed	When onboarding writes it	Touched mappings renewed
Lock / unlock investor	Renewed	Renewed	All supplied mappings renewed; complete wallet set required
Registry queries / direct native transfers	Not renewed	Not renewed	Not renewed

Consequently, a parent record can remain live while an older wallet mapping archives, or token activity can maintain a wallet mapping while its parent archives. OpenZeppelin role entries and native SAC state have separate renewal policies; controller maintenance does not cover them.

Renewal is bounded: each eligible helper call adds at most 30 ledger-days toward a 120-ledger-day target, subject to network limits. One call need not reach the target, while repeated references to a wallet can renew it more than once per transaction. Simulations and failed transactions do not commit renewal.

Impact: No material security impact has been demonstrated. Archived persistent records are recoverable, not deleted; archival does not reset registration, lock state, or authority. Properly prepared transactions can restore the required entries.

Recommended Mitigation: Consider renewing the parent investor alongside touched wallet mappings. Provide bounded, permissionless renewal for explicitly supplied investor/wallet keys, driven by an off-chain index, to cover inactive sibling mappings.

Securitize: Acknowledged.

Cyfrin: Acknowledged.